Winograd Convolution on the CNN Accelerator

How F(2x2, 3x3) works, what we built, how it fits the pipeline, and when it is worth using over direct convolution

CEV FPGA team · cnn-accelerator repository (mnist/) · October 17, 2026

Summary

- **What Winograd does.** F(2x2, 3x3) computes a 2x2 block of 3x3 convolution outputs from a 4x4 input tile with **16 multiplies instead of 36** (2.25x fewer). Everything else is additions, subtractions and constant shifts. In our integer form it is **bit-exact** with direct convolution.
- **What we built.** `winograd_3x3_conv.sv` (tile to 2x2 block, 100,006 vectors pass) and `winograd_weight_transform.sv` (3x3 kernel to 4x4 U', 100,014 vectors pass). Python reference: `winograd_transform_weights()`.
- **Key constraint.** Weights are stored as raw int8 (9 bytes per kernel); U' is computed on chip because storing 12-bit U' as int16 would take 32 bytes per kernel (3.6x).
- **Verdict for today's design** (continuous 1 pixel/clock, one input channel, 32 kernels, 2x2 output, no pooling, KV260): **direct convolution wins**. Winograd saves at most 11% of multipliers and loses that to extra requantizers, buffering and control; with int8 DSP packing, direct uses fewer DSPs outright.
- **Winograd wins** when the pixel source is slower than 2/3 pixel per clock, when the 2x2 max-pool is fused into the convolution, for multi-channel layers (MNIST layer 2, LaneSeg), or with several pixels per clock.

1. How Winograd convolution works

1.1 The idea

A 3x3 convolution computed directly needs 9 multiplies per output. Neighbouring outputs reuse most of the same pixels, and Winograd's minimal filtering algorithm exploits that overlap: it computes a small block of outputs together with fewer multiplies, paying with extra additions. Additions are much cheaper than multiplies in hardware, so the trade can be worth it.

The variant we use is **F(m x m, r x r) = F(2x2, 3x3)**: a 2x2 block of outputs from a 3x3 kernel. It needs an input tile of size (m + r - 1) = **4 x 4**, and exactly $(m + r - 1)^2$ = **16** multiplies.

1.2 One dimension first: F(2, 3)

Two neighbouring outputs of a 3-tap filter g = [g0, g1, g2] need four inputs d0..d3:

```
y0 = g0*d0 + g1*d1 + g2*d2          direct: 6 multiplies
y1 = g0*d1 + g1*d2 + g2*d3
```

Winograd gets the same two outputs with 4 multiplies:

```
m1 = (d0 - d2) * g0
m2 = (d1 + d2) * (g0 + g1 + g2) / 2
m3 = (d2 - d1) * (g0 - g1 + g2) / 2
m4 = (d1 - d3) * g2

y0 = m1 + m2 + m3
y1 = m2 - m3 - m4
```

Why exactly 4 multiplies? Treat the filter as a polynomial $g(x) = g0 + g1\,x + g2\,x^2$. Convolving a 3-tap filter to produce 2 outputs is equivalent to multiplying two small polynomials, and the product has 4 coefficients (degree 3). A degree-3 polynomial is fixed by its values at 4 points, and multiplying polynomials point by point costs one multiply per point. The 4 kernel-side terms are g evaluated at those points:

Point	g(x)	Winograd term
x = 0	g0	u0 = g0
x = 1	g0 + g1 + g2	u1 = (g0 + g1 + g2) / 2
x = -1	g0 - g1 + g2	u2 = (g0 - g1 + g2) / 2
x = infinity (leading coefficient)	g2	u3 = g2

Worked example. g = [1, 2, 3], d = [1, 2, 3, 4]:

```
u  = [1, (1+2+3)/2, (1-2+3)/2, 3]  = [1, 3, 1, 3]
m1 = (1-3)*1 = -2    m2 = (2+3)*3 = 15    m3 = (3-2)*1 = 1    m4 = (2-4)*3 = -6
y0 = -2 + 15 + 1 = 14          direct: 1*1 + 2*2 + 3*3 = 14
y1 = 15 - 1 + 6  = 20          direct: 1*2 + 2*3 + 3*4 = 20
```

1.3 Two dimensions: F(2x2, 3x3)

Nesting the 1-D algorithm over rows and columns gives the matrix form:

```
Y = A^T [ (G g G^T) (.) (B^T d B) ] A          (.) = element-wise multiply
        |---- U ---|    |-- V --|
```

Symbol	Size	What it is	Computed
d	4x4	input tile (pixels)	streamed in
g	3x3	kernel (weights)	trained, stored as int8
$U = G\,g\,G^T$	4x4	transformed kernel	once per kernel
$V = B^T\,d\,B$	4x4	transformed tile	every tile
M = U (.) V	4x4	16 element-wise products: the only real multiplies	every tile
$Y = A^T\,M\,A$	2x2	output transform back to 4 conv results	every tile

```
       | 1   0  -1   0 |            | 1    0    0  |
  B^T = | 0   1   1   0 |       G = | 1/2  1/2  1/2|        A^T = | 1  1   1   0 |
       | 0  -1   1   0 |            | 1/2 -1/2  1/2|              | 0  1  -1  -1 |
       | 0   1   0  -1 |            | 0    0    1  |
```

Figure 1. The F(2x2, 3x3) transform matrices (BT and AT match mnist/scripts/winograd_3x3_vectors.py).

B^T and A^T contain only 0 and +/-1, so the "matrix multiplies" in the input and output transforms are really additions and subtractions; no multiplier is ever built for them. For example, row 0 of B^T d is 1*d[0][c] + 0*d[1][c] - 1*d[2][c] + 0*d[3][c] = d[0][c] - d[2][c], which is exactly the RTL line `bt[0*4+c] = d[0*4+c] - d[2*4+c];`

1.4 The 2G trick: staying in integers

G contains halves, so U is not an integer. We use **G' = 2G** instead:

```
     | 2   0   0 |
G' = | 1   1   1 |       U' = G' g G'^T = 4U        =>   Y' = 4 x (direct result)
     | 1  -1   1 |
     | 0   0   2 |       final step: y >>> 2  (exact, because Y' is always a multiple of 4)
```

The x2 in G' is a 1-bit left shift and the final divide by 4 is a 2-bit arithmetic right shift. Shifts by a constant are pure wiring in hardware (no gates, no delay), so the trick costs nothing and keeps the whole datapath integer and bit-exact.

1.5 Why the tile is 4x4, and why tiles step by 2

Each tile produces a 2x2 output block, so the next tile starts 2 pixels further on. Tiles are 4 wide, so neighbours overlap by 2 columns (and 2 rows). A tile is complete when its bottom-right pixel arrives.

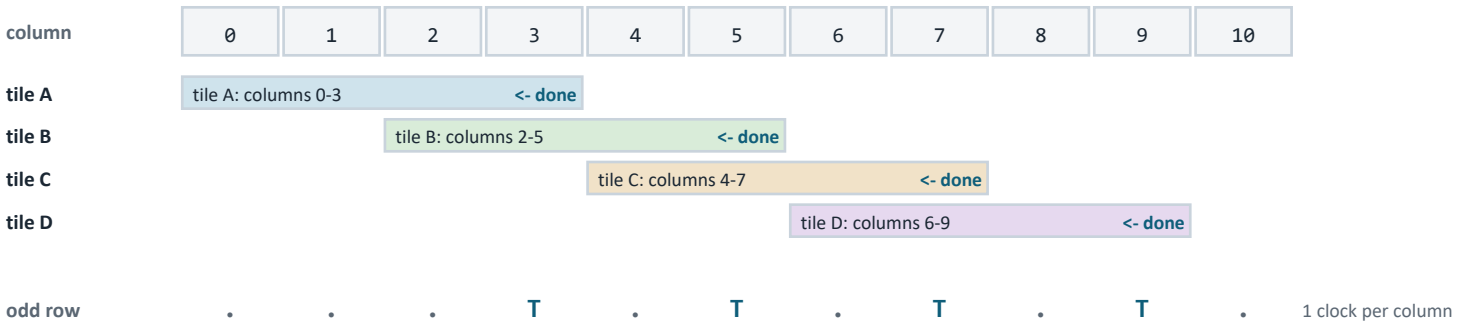


Figure 2. Overlapping 4x4 tiles along one row. A tile completes on columns 3, 5, 7, ... of every odd row (rows 3, 5, 7, ...). At 1 pixel per clock that is one tile every 2 clocks on odd rows and none on even rows.

1.6 Every term written out

Tile pixels are numbered row-major d0..d15 and kernel weights g0..g8.

Input transform $V = B^T d B$ (each entry combines exactly 4 pixels)

```
V0  = d0 - d2  - d8  + d10        V1  = d1 + d2  - d9  - d10
V2  = d2 - d1  - d10 + d9         V3  = d1 - d3  - d9  + d11
V4  = d4 - d6  + d8  - d10        V5  = d5 + d6  + d9  + d10
V6  = d6 - d5  + d10 - d9         V7  = d5 - d7  + d9  - d11
V8  = d8 - d10 - d4  + d6         V9  = d9 + d10 - d5  - d6
V10 = d10 - d9 - d6  + d5         V11 = d9 - d11 - d5  + d7
V12 = d4 - d6  - d12 + d14        V13 = d5 + d6  - d13 - d14
V14 = d6 - d5  - d14 + d13        V15 = d5 - d7  - d13 + d15
```

Kernel transform $U' = G' g G'^T$

```
U'0  = 4*g0                       U'1  = 2*(g0 + g1 + g2)
U'2  = 2*(g0 - g1 + g2)           U'3  = 4*g2
U'4  = 2*(g0 + g3 + g6)           U'5  = g0 + g1 + g2 + g3 + g4 + g5 + g6 + g7 + g8
U'6  = (g0+g3+g6) - (g1+g4+g7) + (g2+g5+g8)
U'7  = 2*(g2 + g5 + g8)
U'8  = 2*(g0 - g3 + g6)
U'9  = (g0-g3+g6) + (g1-g4+g7) + (g2-g5+g8)
U'10 = (g0-g3+g6) - (g1-g4+g7) + (g2-g5+g8)
U'11 = 2*(g2 - g5 + g8)
U'12 = 4*g6                       U'13 = 2*(g6 + g7 + g8)
U'14 = 2*(g6 - g7 + g8)           U'15 = 4*g8

Example, all-ones kernel:   4 6 2 4 / 6 9 3 6 / 2 3 1 2 / 4 6 2 4
```

Multiply, output transform, scale

```
m[i] = U'[i] * V[i]                       i = 0..15  (the 16 real multipliers)

a[c] = m[c] + m[4+c] + m[8+c]             c = 0..3   (A^T m, rows)
b[c] = m[4+c] - m[8+c] - m[12+c]

y0 = a0 + a1 + a2      y1 = a1 - a2 - a3           (... A, columns)
y2 = b0 + b1 + b2      y3 = b1 - b2 - b3

out[k] = y[k] >>> 2                                 out = 2x2 block, row-major
```

No single product m[i] is "a weight times a pixel": each mixes 4 pixels with up to 9 weights, and the output transform cancels the cross terms so that out[0..3] equal the four direct 3x3 results at tile offsets (0,0), (0,1), (1,0), (1,1).

1.7 Why not bigger tiles?

Variant	Tile	Mults / outputs	Mults per output	vs direct (9)	Transform constants
F(2x2, 3x3) (ours)	4x4	16 / 4	4	2.25x	0, +/-1, +/-1/2 (removed by 2G): add/sub only
F(3x3, 3x3)	5x5	25 / 9	2.78	3.24x	needs +/-2: fractions like 1/3, 1/6
F(4x4, 3x3)	6x6	36 / 16	2.25	4x	+/-2: fractions like 1/4, 1/6, 1/24

Larger tiles save more multiplies, but their transforms need constant multiplies and fractions, and the transformed values grow quickly. In int8 that either needs much wider multipliers or introduces rounding error, and it needs deeper line buffers. 4x4 is the largest tile whose transforms are pure add/subtract and stay exact in int8, which is why it is the standard choice for quantized FPGA designs.

2. Our implementation

2.1 Files

File	Role	Status
`mnist/rtl/winograd_3x3_conv.sv`	Combinational F(2x2,3x3): 4x4 tile + 16 U' in, 2x2 block out	100,006 vectors, 0 errors
`mnist/rtl/winograd_weight_transform.sv`	Combinational kernel transform: int8 3x3 in, 12-bit 4x4 U' out	100,014 vectors, 0 errors
`mnist/tb/winograd_3x3_conv_tb.sv`	Vector testbench, expected values from direct convolution	passing
`mnist/tb/winograd_weight_transform_tb.sv`	Vector testbench for the kernel transform	passing
`mnist/scripts/winograd_3x3_vectors.py`	Random + corner tiles/kernels, checks a Python Winograd model against direct conv	used by TB
`mnist/scripts/winograd_weight_transform_vectors.py`	Random + corner kernels and their U'	used by TB
`mnist/scripts/run_tb.ps1`	Generate vectors, compile with Icarus 12, simulate (-Tb, -Regen, -Dump)	working
`mnist/model/quantize_custom.py`	Adds winograd_transform_weights() and export_winograd_mem() (int16 U' export)	tests written, not yet run (no torch on the dev machine)

2.2 winograd_3x3_conv.sv

A purely combinational drop-in for four `convolve_3x3` calls on the overlapping windows of one 4x4 tile.

	convolve_3x3 (direct)	winograd_3x3_conv
Pixel input	pixels [0:8], 8-bit unsigned (3x3)	tile [0:15], 8-bit unsigned (4x4, row-major)
Weight input	weights [0:8], signed 8-bit (raw g)	u_weights [0:15], signed 12-bit (U')
Output	convolution, one signed 20-bit value	convolution [0:3], four signed 20-bit values (2x2)
Multipliers	9 x (9-bit x 8-bit)	16 x (12-bit x 11-bit)
Adders	8 (adder tree)	32 (input transform) + 24 (output transform)
Widest internal	20-bit sum	25-bit (at, y)

```
module winograd_3x3_conv
(
    input  logic         [7:0] tile        [0:15],  // 4x4 pixels, row-major
    input  logic signed [11:0] u_weights   [0:15],  // G' g G'^T, precomputed
    output logic signed [19:0] convolution [0:3]    // 2x2 outputs, row-major
);
    logic signed [10:0] d  [0:15];  // zero-extended pixels
    logic signed [10:0] bt [0:15];  // B^T d
    logic signed [10:0] v  [0:15];  // B^T d B
    logic signed [22:0] m  [0:15];  // U' (.) V
    logic signed [24:0] at [0:7];   // A^T m      (2x4)
    logic signed [24:0] y  [0:3];   // A^T m A    (2x2) = 4 x convolution

    always_comb begin
        ...
        // Input transform, rows: B^T d
        for (int c = 0; c < 4; c++) begin
            bt[0*4+c] = d[0*4+c] - d[2*4+c];
            bt[1*4+c] = d[1*4+c] + d[2*4+c];
            bt[2*4+c] = d[2*4+c] - d[1*4+c];
            bt[3*4+c] = d[1*4+c] - d[3*4+c];
        end
        // Input transform, columns: (B^T d) B   -- same four lines on bt rows
        ...
        // The only 16 multiplies
        for (int i = 0; i < 16; i++) begin
            m[i] = u_weights[i] * v[i];
        end
        // Output transform, rows: A^T m
        for (int c = 0; c < 4; c++) begin
            at[0*4+c] = m[0*4+c] + m[1*4+c] + m[2*4+c];
            at[1*4+c] = m[1*4+c] - m[2*4+c] - m[3*4+c];
        end
        // Output transform, columns: (A^T m) A
        for (int r = 0; r < 2; r++) begin
            y[r*2+0] = at[r*4+0] + at[r*4+1] + at[r*4+2];
            y[r*2+1] = at[r*4+1] - at[r*4+2] - at[r*4+3];
        end
        // U' used 2G, so y is exactly 4x the direct result
        for (int i = 0; i < 4; i++) begin
            convolution[i] = 20'(y[i] >>> 2);
        end
    end
endmodule
```

Listing 1. winograd_3x3_conv.sv (abridged). Each stage maps one-to-one onto the equations in section 1.6.

2.3 winograd_weight_transform.sv

Because we are memory constrained, weights stay stored as raw int8 and U' is produced on chip. In each pass the shared sum (outer + outer) is computed once and reused, so the whole transform is **21 adders** plus free constant shifts. It runs once per kernel load, so a single instance can serve the whole accelerator and it is never on the throughput path.

```
module winograd_weight_transform
(
    input  logic signed  [7:0] weights   [0:8],   // g, 3x3 row-major
    output logic signed [11:0] u_weights [0:15]   // U' = G' g G'^T, 4x4 row-major
);
    logic signed  [9:0] gc [0:2];   // g[0][c] + g[2][c], shared by rows 1 and 2
    logic signed  [9:0] t  [0:11];  // G' g        (4x3), |t| <= 384
    logic signed [11:0] tr [0:3];   // t[r][0] + t[r][2], shared by cols 1 and 2

    always_comb begin
        // Rows: G' g
        for (int c = 0; c < 3; c++) begin
            gc[c]    = weights[0*3+c] + weights[2*3+c];
            t[0*3+c] = weights[0*3+c] <<< 1;
            t[1*3+c] = gc[c] + weights[1*3+c];
            t[2*3+c] = gc[c] - weights[1*3+c];
            t[3*3+c] = weights[2*3+c] <<< 1;
        end
        // Columns: (G' g) G'^T, |U'| <= 1152
        for (int r = 0; r < 4; r++) begin
            tr[r]            = t[r*3+0] + t[r*3+2];
            u_weights[r*4+0] = t[r*3+0] <<< 1;
            u_weights[r*4+1] = tr[r] + t[r*3+1];
            u_weights[r*4+2] = tr[r] - t[r*3+1];
            u_weights[r*4+3] = t[r*3+2] <<< 1;
        end
    end
endmodule
```

Listing 2. winograd_weight_transform.sv (complete).

2.4 Python side

```
WINOGRAD_G2 = torch.tensor([[2, 0, 0], [1, 1, 1], [1, -1, 1], [0, 0, 2]], dtype=torch.int32)

def winograd_transform_weights(weights: torch.Tensor) -> torch.Tensor:
    """Transform int8[out, in, 3, 3] kernels into int16[out, in, 4, 4] U' tiles."""
    g2 = WINOGRAD_G2.to(weights.device)
    u = g2 @ weights.type(torch.int32) @ g2.T
    limit = 2 ** (WINOGRAD_U_BITS - 1)
    assert torch.all((-limit <= u) & (u < limit))
    return u.type(torch.int16)
```

Listing 3. quantize_custom.py: the Python reference for U' (abridged).

`export_winograd_mem()` writes U' into the `.mem` format instead of raw weights. Because 12-bit values need a 2-byte container, a kernel grows from 9 bytes to 32 bytes (3.6x). Given the memory constraint, the plan is to keep using the existing `export_mem()` (raw int8) and the on-chip transform; the Python function remains the golden reference.

2.5 Verification

Testbench	Vectors	What is checked	Result
winograd_3x3_conv_tb	100,000 random + 6 corner	4 outputs vs direct 3x3 convolution of the 4 windows	PASS, 0 errors
winograd_weight_transform_tb	100,000 random + 14 corner	all 16 U' values vs G' g G'^T	PASS, 0 errors
test_quantize_custom.py (new tests)	random + extremes, 3 input channels	U' (.) V summed over channels, one output transform, equals direct	written, not yet run

```
powershell -ExecutionPolicy Bypass -File mnist\scripts\run_tb.ps1 -Tb winograd_3x3_conv
powershell -ExecutionPolicy Bypass -File mnist\scripts\run_tb.ps1 -Tb winograd_weight_transform
uv run python -m pytest mnist/model/test_quantize_custom.py
```

3. The data path

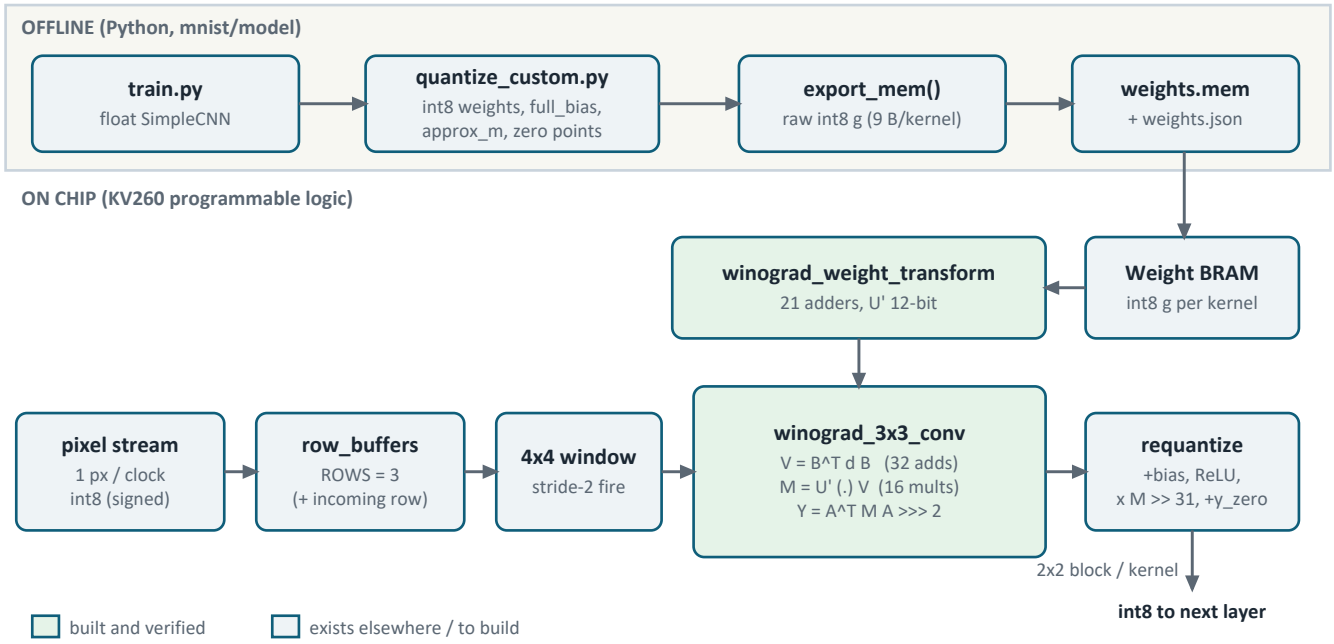


Figure 3. Layer-1 data path with Winograd. Weights are trained and quantized offline, stored on chip as raw int8, transformed once per kernel load, and combined with the transformed pixel tiles.

3.1 Status of each block

Block	File	Status
Training, quantization, bit-exact integer reference	model/quantize_custom.py	Done
Raw int8 export (.mem + .json)	export_mem()	Done
Weight BRAM loaded from .mem	-	Not built
Kernel to U' transform	winograd_weight_transform.sv	Done, verified
Row buffers	row_buffers.sv	Done (ROWS is a parameter)
4x4 window, stride-2 firing, padding with x_zero	(winograd FSM)	Not built
Winograd convolution	winograd_3x3_conv.sv	Done, verified; needs signed pixels
Bias + ReLU + requantization	quantized_scale.sv	Needs rework (being handled separately)
Fused 2x2 max-pool	-	Not built
Multi-channel layer 2, FC layer, top level, end-to-end test	-	Not built

3.2 What the quantized model requires

`QuantizedConvRelu2d` in quantize_custom.py is the integer reference the hardware must match bit for bit. It imposes these requirements on the convolution datapath (direct or Winograd):

Requirement	Detail	Impact on Winograd
Signed int8 activations	Every layer input is int8 with a zero point; MNIST input has x_zero = -128	tile port must sign-extend; V shrinks to 10 bits (-512..510)
Zero point folded into bias	full_bias = bias - sum(w) * x_zero	feed raw q; no per-pixel subtraction
Padding = x_zero	Conv2d(padding=1) pads with the input zero point	FSM must inject x_zero at borders
Symmetric weights	weight zero point is always 0	nothing extra
Per-channel requantization	approx_m (31 fraction bits) per output channel, round half up, + y_zero, clip	one multiply per requantizer
int32 accumulation	multi-channel sums exceed 20 bits	widen accumulators for layer 2
Conv then ReLU then MaxPool2d(2)	both conv layers are followed by a 2x2 max-pool	the 2x2 Winograd block is exactly one pool window

3.3 Bit widths and overflow

U' cannot overflow: rows of G' sum to at most 3 in absolute value, so $|U'| \le 3 \times 3 \times 128 = 1152$, inside the 12-bit range (-2048..2047). `winograd_transform_weights()` asserts this too.

Signal	Worst case (signed int8 pixels)	Bits needed	Current RTL
V	4 x 128 = 512	10-11	11
m = U' x V	1152 x 512 = ~590k	21	23
A^T m (first pass)	3 x 590k = ~1.77M	22	25
y (= 4 x direct)	4 x 9 x 128 x 128 = ~590k	21	25
convolution (y >>> 2)	147,456	19	20
Layer 2 (C = 8) sums	+ log2(C) = +3 bits; plus bias	24-26	needs widening (use 32)

4. Winograd in a pixel-streaming design

The existing `convolve_3x3_fsm.sv` takes one pixel per clock, keeps 2 previous rows in `row_buffers`, slides a 3x3 window, and runs `KERNEL_COUNT` (default 32) `quantized_convolution` instances in parallel: one output per kernel per clock, 9 x 32 = 288 multipliers.

4.1 Tiles arrive in bursts

A direct 3x3 window steps by 1, so every pixel completes a window. Winograd tiles step by 2 in both directions, so tiles complete only on odd rows (3, 5, 7, ...) at odd columns (3, 5, 7, ...):

```
even row:  p p p p p p p p p ...     no tile completes (top half of the next tile row)
odd row:   p p p T p T p T p T ...   one tile every 2 clocks
average:   1 tile per 4 clocks  =  4 outputs per 4 pixels  =  same output rate as direct
```

4.2 Compute time versus arrival gap

There are two different "2 clocks" in this design. The **arrival gap** between tiles is set by the input (2 clocks at 1 pixel/clock). The **compute time** is 16 / (multipliers per kernel). The hardware can never go faster than the pixels arrive.

Multipliers / kernel	Compute time	Arrival gap	Behaviour	DSPs (32 kernels)
16	1 clock	2 clocks	idle half the time on odd rows, all of even rows (25% busy)	512
8	2 clocks (m0..m7, then m8..m15)	2 clocks	exactly keeps up; idle on even rows (50% busy)	256
4	4 clocks	2 clocks	falls behind: needs pixel_ready backpressure, 2/3 pixel/clock average	128

8 multipliers per kernel is the minimum for a continuous 1 pixel/clock stream without backpressure. Going below 8 needs either stalls or a buffer that spreads the odd rows' tiles over the even rows (about a row of transformed tiles, ~28 kbit at width 320).

4.3 Requantizers double

A requantizer handles one value per clock. Direct produces 1 value per kernel per clock. Winograd produces 4 values per tile every 2 clocks on odd rows: **2 per clock at peak** (0 on even rows, 1 on average). Peak rate decides the hardware, and it depends on the tile arrival rate, not the multiplier count, so even 16 multipliers per kernel does not help.

Option	Requantizers per kernel
Direct	1
Winograd, quantize all 4 in one clock	4
Winograd, 2 per clock over the tile's 2 clocks	2
Winograd, defer to the even row	1, but buffers ~40 KB of unquantized 32-bit results
Winograd with fused 2x2 max-pool (max first, then quantize; bit-identical)	1/2 (one per two kernels)

4.4 Output order and latency

- **Order.** Direct emits raster order. Winograd emits 2x2 blocks covering two output rows at once. A raster consumer needs a reorder buffer of one output row per kernel: 32 x 318 bytes = ~10 KB.
- **Latency.** First output after row 3, column 3 instead of row 2, column 2 (about one extra row).
- **Line buffers.** 3 stored rows instead of 2 (+320 bytes at width 320).
- **Critical path.** add, add, multiply, add, add, add in one combinational block; needs pipeline registers at 200+ MHz.

5. Constraints and cost on the KV260

5.1 The target

The KV260 uses the Kria K26 module (Zynq UltraScale+): approximately **1,248 DSP48E2** slices, ~117k LUTs, 144 block RAMs (36 Kb) and 64 UltraRAMs (~23 Mb on-chip total), plus a quad-core ARM processing system with 4 GB DDR4.

5.2 Operand widths and DSP packing

- Each DSP48E2 has one 27 x 18 multiplier. A Winograd 12 x 11 (or 12 x 10 with signed pixels) multiply fits in one DSP; so does a direct 9 x 8.
- **Int8 packing:** two 8-bit multiplies that share one operand can be packed into one DSP48E2. In direct convolution every kernel multiplies the same pixel, so two kernels can share a DSP: 288 multiplies on 144 DSPs. Packing must be written explicitly in RTL.
- **Winograd cannot pack:** two ~23-bit products of 12-bit U' values do not fit side by side in the 27-bit port. The wider operands also cost more if multipliers are built from LUTs (16 x 132 partial-product bits vs 36 x 64: roughly equal).

5.3 Head to head: continuous 1 pixel/clock, 1 input channel, 32 kernels

	Direct (current FSM)	Winograd, 8 / kernel
Multipliers	288 (9x8)	256 (12x11)
DSP48E2	288, or **144 with int8 packing**	**256**
Input transform	none	32 adders, shared by all kernels
Weight transform	none	21 adders per kernel (or 21 shared with serial loading)
Output adders	8 per kernel	24 per kernel + muxes/registers for 2-clock sharing
Requantizers	32	64
Line buffers	2 rows	3 rows
Window registers	9 bytes	16 bytes
Output	raster, 1 value / kernel / clock	2x2 blocks, odd rows only; ~10 KB reorder for raster
Throughput	1 pixel / clock	1 pixel / clock
Latency	after row 2, col 2	after row 3, col 3
Control	simple	stride-2 firing, 2-clock sequencing, burst output
Accuracy	exact	exact (identical results)

Result: for the current configuration direct convolution wins. Winograd saves at most 32 DSPs (11%) against unpacked direct and gives most of it back in requantizers, buffering and control. Against direct with int8 packing it uses 1.8x more DSPs.

5.4 Memory

Item	Direct	Winograd
Weights per kernel (raw int8 + on-chip transform)	9 B	9 B
Weights per kernel (stored U' as int16)	-	32 B (3.6x): rejected
Line buffers (width 320, 1 channel)	640 B	960 B
Reorder buffer for raster output (32 kernels)	0	~10 KB (0 with fused pool)
Deferred-requantization buffer (if used)	0	~40 KB

6. Arguments against Winograd (for the current design)

1. **The input sets the pace.** At 1 pixel/clock both methods produce one output per pixel. Winograd's fewer multiplies only matter if the multipliers can be reused, and with bursty tiles they sit idle half or more of the time.
2. **Wider multipliers.** 12 x 11 instead of 9 x 8. On Intel-style DSPs that halves how many fit per block; on DSP48E2 it rules out int8 packing, which gives direct a 2x DSP advantage on its own.
3. **Bursty output.** Peak output rate is 2 values per kernel per clock, which doubles the requantizers. Once requantization uses a runtime per-channel multiplier, that is real DSPs.

4. **Not raster order.** 2x2 blocks need a reorder buffer (~10 KB) unless the next stage consumes blocks (a pool does).
5. **More buffering and latency.** One extra row buffer, a bigger window, about one row of extra latency.
6. **More complex control.** Stride-2 firing, multiplier sequencing, burst handling, and edge handling for odd output sizes.
7. **Timing.** A longer combinational chain that needs pipelining.
8. **Only exact for small tiles.** Bigger tiles (which would save more) break int8 exactness.

7. When Winograd would be useful

Situation	Why it helps	Effect
Pixel source slower than 2/3 pixel/clock	4 multipliers per kernel never stalls; pixel_ready stays high	128 DSPs vs 288 (144 packed)
Fused 2x2 max-pool	One value per tile: no reorder buffer, 1 requantizer per 2 kernels, plain raster output; max-then-quantize equals quantize-then-max because ReLU and requantization are monotonic	removes most integration cost
Multi-channel layers (MNIST layer 2, LaneSeg)	V computed once per tile and reused by every filter; products summed over channels in the Winograd domain; one output transform per filter	transforms become small overhead; close to the full 2.25x multiply saving
Several pixels per clock (compute bound)	A new tile every clock keeps 16 multipliers per kernel busy	2.25x fewer multiplies; ~1.1x fewer DSPs vs packed direct

7.1 The slow-source case is likely for LaneSeg

With a 200 MHz fabric clock, a 320x240 camera at 30 frames/s delivers about 2.3 million pixels/s: roughly **one pixel every 87 clocks**. MNIST at 1,000 images/s is one pixel every ~250 clocks. With that much slack, the 4-multiplier Winograd design never stalls, and one set of multipliers could even loop over several kernels or input channels.

7.2 Which LaneSeg layer types benefit

Layer type	Winograd?
3x3, stride 1, many channels	Yes, ~2.25x fewer multiplies (usually most of the compute)
3x1 / 1x3 factorized (ERFNet style)	Yes, 1-D F(2,3): 4 multiplies instead of 6 (1.5x)
Dilated 3x3	Yes, on every d-th pixel; tile spans more rows, so bigger line buffers
3x3, stride 2 (downsampling)	Not really: decomposes into sub-convolutions and the gain disappears
1x1 / pointwise	No gain
Depthwise 3x3	Little: no reuse across channels
Transposed convolution (decoder upsampling)	Possible but complex; use direct

Rule of thumb on the KV260: Winograd pays off when the multipliers would otherwise sit idle or be replicated (slow source, many channels) **and** the output is pooled right away. At full 1 pixel/clock on a single channel, direct convolution with int8 packing is simpler and cheaper.

8. Recommendations and next steps

1. **Keep the direct FSM** for the current single-channel, continuous 1 pixel/clock layer; add explicit int8 DSP packing if DSPs become tight.
2. **Use Winograd where it wins:** a multi-channel unit (layer 2) with the 2x2 max-pool fused in, built from the verified winograd_weight_transform and the transform stages of winograd_3x3_conv (split into input transform, per-channel multiply-accumulate, and one output transform per filter).
3. **Common fixes needed either way:** signed int8 pixel inputs, padding with x_zero, bias and per-channel requantization matching QuantizedConvRelu2d, 32-bit accumulators for multi-channel layers, FC weight reordering in the export.
4. **If a slow source is confirmed** (e.g. LaneSeg camera), build the Winograd FSM with a ready/valid (AXI-Stream style) input and a MULT_LANES parameter (4 / 8 / 16).
5. **Verify end to end** against QuantizedConvRelu2d using a real exported .mem file.